

Design Patterns

Atividade Avaliativa de Engenharia de Software

Dupla: Jaime Willian Carneiro da Silva e Luiz Eduardo Bravin

1. Configuração da aplicação

1. O Python invoca `__init__` automaticamente sempre que `__new__` devolve uma instância da classe. Como o `__new__` devolve a mesma instância em chamadas subsequentes, o `__init__` é executado novamente.
2. Como os módulos em Python são executados apenas na primeira importação e ficam em cache em `sys.modules`, definir variáveis globais diretamente num ficheiro `.py` cria um estado partilhado acessível via import em toda a aplicação sem recurso a classes.
3. Cria dependências ocultas e efeitos colaterais entre partes do código, o que prejudica o isolamento dos testes unitários, visto que a mutação num teste afeta os seguintes.

2. Pedido e Builder

1. **Builder:** `OrderBuilder`. **Objeto construído:** `Order`.
2. Embora o Python suporte parâmetros com valores default e por palavra-chave (kwargs), tornando a instanciação direta viável, a diferença é que o construtor direto gera assinaturas extensas e não permite validação gradual, enquanto o Builder separa a montagem passo a passo da validação das regras (como a obrigatoriedade do cliente) antes de consolidar a instância final.

3. Pagamento e Factory Method

1.
 - **Creator:** `PaymentProcessor`
 - **Concrete Creator:** `PixProcessor` , `CreditCardProcessor` e `BoletoProcessor`
 - **Product:** `Payment`
 - **Concrete Product:** `PixPayment` , `CreditCardPayment` e `BoletoPayment`
2. Uma cadeia de `if/elif` caracteriza apenas uma fábrica simples. O padrão Factory Method fundamenta-se em polimorfismo e herança: a classe base define a operação com um método abstrato de criação e as subclasses decidem qual classe concreta instanciar.
3. Nenhuma classe existente precisa de ser alterada. Apenas se estende o sistema criando uma nova subclasse de `Payment` e uma nova subclasse de `PaymentProcessor` , cumprindo o princípio Open/Closed do SOLID.

4. Famílias por canal

1. Porque ambos variam juntos em função do contexto do canal de venda (WEB, MOBILE ou KIOSK), exigindo interfaces e comportamentos correspondentes e coerentes entre si na mesma plataforma.
2. Garante a criação de objetos compatíveis de uma mesma família sem misturar implementações.
3. Porque canal e forma de pagamento variam de forma independente. Acoplar o pagamento à fábrica do canal violaria o princípio de responsabilidade única (SRP).

5. Seleção de fábrica e alteração do sistema

1. **Arquivos criados:** `channels/kiosk.py` , contendo `KioskCheckout` , `KioskNotification` e `KioskFactory` . Nenhum ficheiro do sistema foi alterado.
2. São compatíveis com o OCP porque o sistema foi estendido com novas classes sem modificar o código de `get_channel_factory` , das abstrações base ou dos canais existentes, permanecendo fechado para alteração e aberto para extensão.

6. Responsabilidades e integração

1. A classe coordenadora `OrderService` tem como responsabilidade principal apenas orquestrar o fluxo, chamando as demais classes e funções na ordem correta.
 - **AppConfig:** Armazenar as informações do ambiente e garantir o acesso a uma única instância global compartilhada.
 - **OrderBuilder:** Isolar a complexidade de criar os pedidos passo a passo, lidando com a validação de atributos obrigatórios e a adição de opcionais.
 - **OrderService:** Orquestrar o fluxo principal delegando a execução aos componentes adequados, sem acoplar-se a implementações concretas.
 - **PaymentProcessor:** Atua como componente base do fluxo de cobrança, delegando a criação da forma de pagamento exata para as subclasses. Isola a lógica de processamento.
 - **get_channel_factory:** Selecionar e retornar qual factory deve ser usada para o canal informado.
 - **Channel Factory (e filhas):** Agrupam a criação do checkout e da notificação, garantindo que os objetos criados pertençam à mesma família.
 - **Order e Product:** Guardam os dados essenciais e as regras básicas do negócio.
 - **Payment (e filhas):** Executam a ação específica daquela forma de pagamento.
 - **Checkout e Notification:** Isolam a lógica visual ou de envio de mensagens de cada canal.
2. Focando em `OrderBuilder`, `AppConfig` e `PixPayment` :
 - **OrderBuilder:** Se a regra de negócio mudar e exigir que "endereço" seja obrigatório, a validação muda apenas no método `.build()` do Builder.
 - **AppConfig:** Se no futuro precisar de uma variável nova, adiciona-se diretamente no AppConfig. O resto do sistema não sofrerá alteração.
 - **PixPayment:** Se a API do banco passar a exigir um novo parâmetro, a mudança será feita diretamente no `PixPayment`.
3. Atualmente os canais são registrados escrevendo as chamadas diretamente no código-fonte. Uma alternativa seria o registro dinâmico lendo a lista de canais disponíveis a partir de um arquivo externo (JSON ou `.env`). Isso aumentaria a flexibilidade, permitindo adicionar novos canais (como o KIOSK) apenas inserindo uma linha nesse arquivo, sem alterar o código original.

7. Testes e alterações

1. `test_double_config` :

- Gerar dois `AppConfig` resulta em uma única instância.
- Se as instâncias forem idênticas, o teste passa silenciosamente e a função não retorna nada.
- Se o sistema permitisse instâncias diferentes, teríamos inconsistências graves, pois cada módulo poderia usar parâmetros distintos, gerando erros imprevisíveis.

2. `test_builder_empty_list` :

- Criação de um pedido fornecendo apenas o cliente, sem adicionar produtos.
- O resultado esperado deve ser o `ValueError`: *"O pedido deve conter pelo menos um produto"*.
- Previne erros de execução (como `TypeError`) caso outras partes do sistema tentem iterar sobre os produtos de um pedido vazio.

3. `test_different_kiosk_factory_checkout_instances` :

- Criação de múltiplos objetos Checkout pela mesma fábrica.
- Cada chamada retorna uma instância completamente diferente e independente na memória.
- Garante o isolamento dos dados, evitando que transações simultâneas se misturem.

8. Situação de mudança

Nova forma de pagamento escolhida: Cartão de Débito

1. Os arquivos modificados foram: `payments/methods.py` e `payments/processors.py`.
2. O fluxo principal não precisou ser alterado.
3. Nenhuma classe existente precisou ser alterada.
4. Com o Factory Method, há uma classe abstrata de `PaymentProcessor` e, para cada forma de pagamento nova, basta herdar dessa classe. O sistema está aberto para expansão e fechado para modificação.

9. Organização e Estrutura dos Módulos

A estrutura do projeto adota uma arquitetura modular baseada nos princípios de Responsabilidade Única (SRP) e Aberto/Fechado (OCP) para garantir baixo acoplamento e alta coesão. O objetivo central é isolar as regras de negócio do domínio de detalhes de infraestrutura, garantindo que o núcleo do sistema não dependa de canais de atendimento ou métodos de pagamento específicos.

Para viabilizar isso, as responsabilidades foram distribuídas da seguinte forma:

- **channels/** : Isola as interfaces de interação com o usuário, permitindo o registro dinâmico de novos canais sem alterar o código existente.
- **payments/** : Centraliza as regras financeiras e padroniza contratos, facilitando a adição de novos meios de cobrança.
- **domain/** : Concentra as entidades centrais do negócio, mantendo a regra principal isolada e independente de tecnologias externas.
- **services/** : Foca na orquestração dos fluxos da aplicação, coordenando a comunicação entre o domínio e as demais integrações.
- **config/** : Mantém as configurações globais do sistema separadas da lógica de produção.
- **tests/** : Agrupa os arquivos e cenários de testes automatizados, mantendo-os fora do código de produção.
- **main.py** : Atua estritamente como o ponto de entrada para inicializar e orquestrar a integração dos componentes do sistema.
- **tests.py** : Serve como o executável central responsável exclusivamente por acionar as baterias de testes.